LAB 10: MEMORY MANAGEMENT AND ADDRESS TRANSLATION

Objective:

Understand and implement key memory management techniques in Operating Systems, including dynamic memory allocation, paging, and address translation.

Demonstrate the integration of semaphores for synchronized access to memory resources.

Explore how address translation maps logical addresses to physical ones in a paged memory system.

Develop hands-on understanding of allocation, synchronization, and logical-to-physical mapping processes using C programming.

Experiment 1: Memory Management

Memory management is an essential function of the operating system that handles the allocation and deallocation of memory space to processes efficiently. The goal is to optimize the use of RAM and ensure process isolation.

1. Memory Management Techniques:

- **Contiguous Allocation:** Each process occupies a single contiguous block of memory.
- **Paging:** Divides physical memory into fixed-size blocks called frames and logical memory into pages.
- **Segmentation:** Divides processes into variable-sized segments.
- **Dynamic Allocation:** Allocates memory at runtime as per the program's requirements using functions such as `malloc()`, `calloc()`, and `free()`.

2. Paging Example:

Paging removes external fragmentation and allows non-contiguous allocation of process memory.
A **Page Table** maps logical pages to physical frames, and address translation is performed using:

```
Physical Address = (Frame Number × Frame Size) + Offset
```

Algorithm:

1. Input total number of available frames.
2. Initialize used frames to zero.
3. For each process, input required pages.
4. If enough frames are available, allocate memory and update used frames.
5. Otherwise, display memory allocation failed message.
6. Repeat until the user enters 0 to stop.
7. Display final memory usage and end the program.

Program:

```
#include <stdio.h>


int main() {
    int total_frames, used_frames = 0;
    int process_pages, remaining_frames;

    printf("Enter total number of available frames in memory: ");
    scanf("%d", &total_frames);

    while (1) {
        printf("\nEnter number of pages required by process (0 to stop): ");
```

```
        scanf("%d", &process_pages);


        if (process_pages == 0)
            break;


        remaining_frames = total_frames - used_frames;


        if (process_pages <= remaining_frames) {
            printf("Memory allocated successfully to process.\n");
            used_frames += process_pages;
            printf("Total used frames: %d | Free frames: %d\n", used_frames,
total_frames - used_frames);
        } else {
            printf("Memory allocation failed! Not enough free frames.\n");
            printf("Available frames: %d | Requested: %d\n", remaining_frames,
process_pages);
        }
    }


    printf("\nAll processes handled. Final memory usage: %d/%d frames used.
\n", used_frames, total_frames);
    return 0;
}
```

Explanation:

This experiment simulates **dynamic memory management** where each process requests a certain number of pages.

The system checks if enough free frames are available and allocates them sequentially.

If memory is insufficient, the process is denied allocation which representing **memory fragmentation or insufficient memory** conditions.

Sample Output:

Enter total number of available frames in memory: 10

Enter number of pages required by process (0 to stop): 4

Memory allocated successfully to process.

Total used frames: 4 | Free frames: 6

Enter number of pages required by process (0 to stop): 3

Memory allocated successfully to process.

Total used frames: 7 | Free frames: 3

Enter number of pages required by process (0 to stop): 5

Memory allocation failed! Not enough free frames.

Available frames: 3 | Requested: 5

Enter number of pages required by process (0 to stop): 2

Memory allocated successfully to process.

Total used frames: 9 | Free frames: 1

Enter number of pages required by process (0 to stop): 0

All processes handled. Final memory usage: 9/10 frames used.

Program 2:

```
#include <stdio.h>


int main() {
    int page_table[10], pages, page_size;
    int logical_addr, page_no, offset, physical_addr;

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    printf("Enter page size (bytes): ");
    scanf("%d", &page_size);

    printf("Enter frame number for each page:\n");
    for (int i = 0; i < pages; i++) {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &page_table[i]);
    }

    printf("Enter logical address: ");
    scanf("%d", &logical_addr);

    page_no = logical_addr / page_size;
    offset = logical_addr % page_size;

    if (page_no >= pages)
        printf("Invalid logical address!\n");
```

```
    else {
        physical_addr = (page_table[page_no] * page_size) + offset;
        printf("Physical Address = %d\n", physical_addr);
    }

    return 0;
}
```

Sample Output:

Enter number of pages: 4

Enter page size (bytes): 100

Enter frame number for each page:

Page 0 -> Frame: 5

Page 1 -> Frame: 9

Page 2 -> Frame: 3

Page 3 -> Frame: 7

Enter logical address: 250

Physical Address = 950

Conclusion:

Paging supports efficient memory management by allowing non-contiguous allocation, improving memory utilization and access flexibility.

Memory Management with Integrated Semaphore

In multi-threaded or multiprogramming systems, multiple processes may request or release memory at the same time.
This can lead to **race conditions** or data inconsistency if multiple threads access the same memory concurrently.
To prevent this, **semaphores** are introduced as synchronization primitives to ensure only one process accesses the critical section (memory allocation) at a time.

Semaphore Integration Concept:

1. Initialize semaphore using `sem_init(&mutex, 0, 1)`.
2. Use `sem_wait(&mutex)` before memory allocation to enter the critical section.
3. Perform memory allocation, usage, and deallocation.
4. Use `sem_post(&mutex)` after freeing memory to release the lock.

Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>


sem_t mem_lock; // Semaphore to protect memory access


void *allocate_memory(void *arg) {

    int *id = (int *)arg;

    sem_wait(&mem_lock);  // Enter critical section


    printf("\n[Process %d] Allocating memory...\n", *id);

    int *arr = (int *)malloc(3 * sizeof(int));


    if (arr == NULL) {

        printf("[Process %d] Memory allocation failed.\n", *id);

        sem_post(&mem_lock);

        pthread_exit(NULL);

    }
```

```
    for (int i = 0; i < 3; i++)
        arr[i] = (*id) * 10 + i;

    printf("[Process %d] Stored values: ", *id);
    for (int i = 0; i < 3; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    printf("[Process %d] Memory freed successfully.\n", *id);
    sem_post(&mem_lock);  // Exit critical section
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    sem_init(&mem_lock, 0, 1);  // Initialize semaphore

    pthread_create(&t1, NULL, allocate_memory, &id1);
    pthread_create(&t2, NULL, allocate_memory, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&mem_lock);
```

```
    return 0;
}
```

Explanation:

- Two threads simulate two processes requesting memory concurrently.
- The **semaphore (**`mem_lock`**)** ensures mutual exclusion in the memory allocation section.
- Only one thread can allocate and free memory at a time, avoiding race conditions.

Expected Output:

```
[Process 1] Allocating memory...
[Process 1] Stored values: 10 11 12
```

[Process 1] Memory freed successfully.

[Process 2] Allocating memory...

[Process 2] Stored values: 20 21 22

[Process 2] Memory freed successfully.

Conclusion:

The integration of semaphores with memory management ensures that critical memory operations are performed in a synchronized and safe manner, preventing concurrent access issues in multiprogramming environments.

Experiment 2: Address Translation

When a process executes, it generates **logical (virtual) addresses**, not physical ones.
The **Operating System and Memory Management Unit (MMU)**

translate these logical addresses into **physical addresses** using the **page table**.

- **Logical Address = (Page Number, Offset)**
- **Physical Address = (Frame Number × Frame Size) + Offset**

This translation allows multiple processes to share physical memory while maintaining isolation and protection.

Algorithm / Procedure:

1. Start the program.
2. Input the number of pages, page size, and page table mapping.
3. Accept a logical address as input.
4. Divide the logical address into **page number** and **offset**.
5. Look up the page number in the **page table** to find the frame number.
6. Compute the **physical address** using:

Physical Address=(Frame Number×Page Size)+Offset

7. Display the physical address.

Program:

```
#include <stdio.h>


int main() {
    int page_table[10], i, pages, page_size;
    int logical_addr, page_no, offset, physical_addr;

    // Input total number of pages and page size
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter page size (bytes): ");
```

```
scanf("%d", &page_size);

// Build the page table mapping
printf("Enter frame number for each page:\n");
for (i = 0; i < pages; i++) {
    printf("Page %d -> Frame: ", i);
    scanf("%d", &page_table[i]);
}

// Input logical address to translate
printf("\nEnter logical address: ");
scanf("%d", &logical_addr);

// Calculate page number and offset
page_no = logical_addr / page_size;
offset = logical_addr % page_size;

// Check validity of the logical address
if (page_no >= pages)
    printf("Invalid logical address!\n");
else {
    // Perform address translation using the page table
    physical_addr = (page_table[page_no] * page_size) + offset;
    printf("Physical Address = %d\n", physical_addr);
}
```

```
    return 0;
}
```

Code Explanation Summary:

- The **page table** stores frame numbers for each page.
- Logical address is **divided** into *page number* and *offset* using division and modulus.
- The **physical address** is then computed using the mapping formula.
- This simulates how the **MMU translates logical to physical addresses** in an actual OS.

Expected Output :

Enter number of pages: 4

Enter page size (bytes): 100

Enter frame number for each page:

Page 0 -> Frame: 5

Page 1 -> Frame: 9

Page 2 -> Frame: 3

Page 3 -> Frame: 7

Enter logical address: 250

Physical Address = 350

Conclusion:

The program successfully demonstrates **logical-to-physical address translation** using paging. The mapping through a **page table** enables virtual memory implementation and process isolation in modern operating systems.

Graded Tasks

Task 1: Memory Management with Integrated Semaphore

In this task, students will simulate **dynamic memory allocation** with **synchronization control** using semaphores.
The program should create **three concurrent threads**, each representing a process that dynamically allocates a user-defined number of integers in memory.
To demonstrate synchronization, one thread should deliberately delay the deallocation step using `sleep()` or a computational loop, allowing observation of the **critical section lock** managed by the semaphore.

The system should ensure that only one process accesses the memory allocation section at a time, preventing overlapping or inconsistent memory operations.
At the end of execution, display each thread's allocated memory block and confirm that all deallocations are safely handled without data corruption.

Task 2: Address Translation Simulation

In this task, students will design and implement a **paging-based address translation simulator** that maintains a **page table** and a **free frame list** to manage memory efficiently.
The system will use fixed-size frames and load the free frame list from memory (or a file). The process size will be user-defined, based on which the program determines the number of required pages and assigns them to unique frames.

After allocation, the generated page table should be used for translating **multiple logical addresses** (provided by the user) into **physical addresses**.
Each logical address translation should clearly show the corresponding **page number**, **frame number**, and **offset**.
The final output must include:

- The generated page table
- Remaining free frames in memory
- The list of logical addresses with their translated physical addresses